

UF4

Alternativas en la creación de nuevos componentes

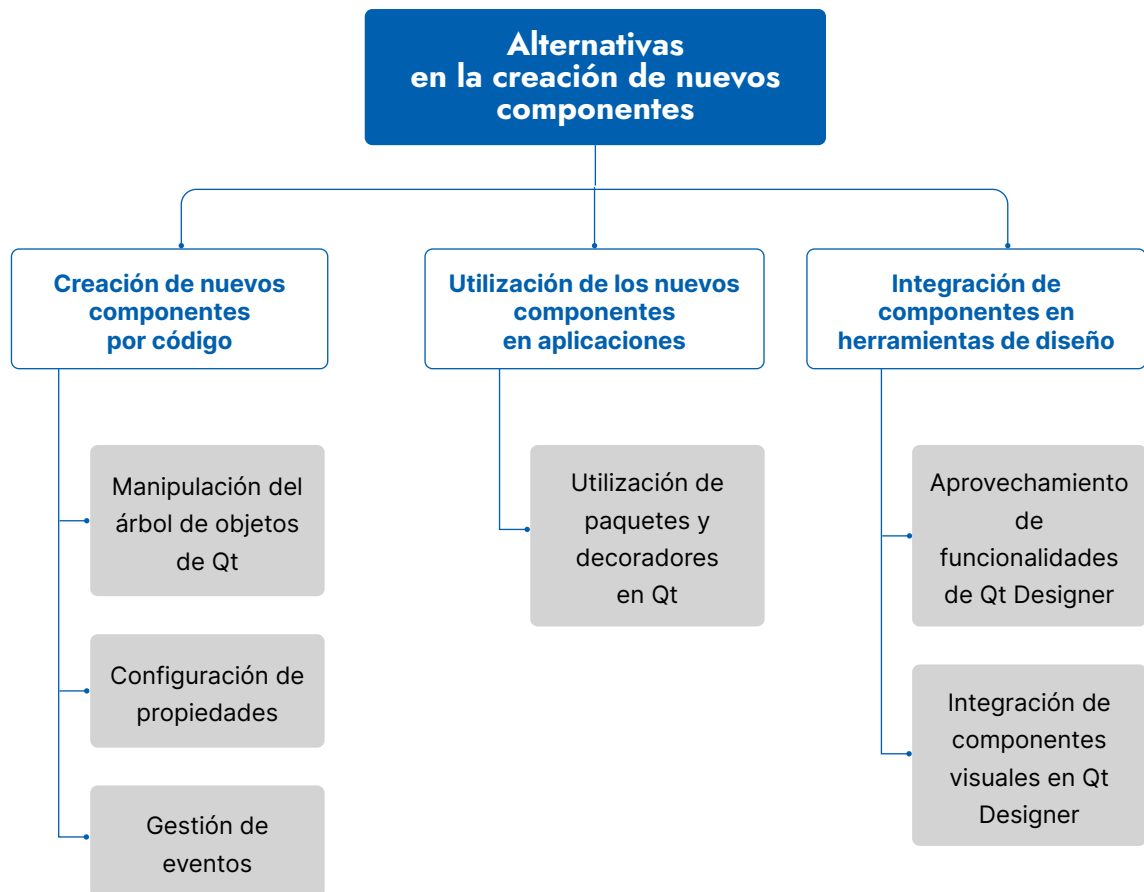
Desarrollo de interfaces

ÍNDICE

Mapa conceptual	04
1. Alternativas en la creación de nuevos componentes.....	05
1.1. Personalizar la apariencia del componente	05
1.2. Derivar de un componente existente	06
1.3. Crear un componente completamente nuevo	06
1.4. Agregar varios componentes existentes	07
2. Creación de nuevos componentes por código.....	07
2.1. Componentes definidos por el usuario.....	07
2.2. Ejemplo: componente cronómetro	08
A. Elección de objeto base.....	08
B. Clase cronómetro (sin interfaz).....	08
C. Definición de la interfaz del componente	09
D. Señales definidas por el usuario	10
E. Lógica interna del componente.....	11
3. Utilización de los nuevos componentes en aplicaciones	13
3.1. Importación de componentes	13
A. Módulos.....	13
B. Paquetes	14
3.2. Uso de componentes en la aplicación principal	16
B. Uso de la señal del componente.....	17
3.3. Uso de decoradores en Python.....	18
A. Decoradores	18
B. El decorador @SLOT().....	19
C. El decorador @PROPERTY	20

4. Integración de componentes en herramientas de diseño	22
4.1. Promoción de componentes.....	23
A. Promoción del componente en QT Designer	23
B. Uso de la interfaz diseñada en una aplicación	24
4.2. Creación de complementos (plugins)	25
A. Actualizar la variable de entorno <i>PYSIDE_DESIGNER_PLUGINS</i>	25
B. Crear el script de registro para el plugin	26
C. Uso de la interfaz diseñada en una aplicación.....	27

Mapa conceptual



01 Alternativas en la creación de nuevos componentes

Todas las tecnologías destinadas a la creación de interfaces de usuario incluyen un **conjunto de componentes para utilizar en los formularios diseñados**, formado por los controles más habituales en las **aplicaciones GUI**. Sin embargo, en algunas ocasiones la funcionalidad de nuestra aplicación puede requerir el uso de un componente que nuestra tecnología de interfaz no incluye.

Para incorporar nuevos componentes a nuestras aplicaciones, tenemos dos alternativas:

» **Utilizar componentes desarrollados por terceros (empresas o desarrolladores independientes)**. Es importante conocer la licencia que aplica el componente, sobre todo si se trata de una licencia comercial o libre.

» **Desarrollar el componente que necesitamos para utilizarlo en nuestra aplicación.**

La mayoría de las tecnologías de interfaz de usuario incorporan mecanismos que permiten a los desarrolladores crear nuevos componentes.

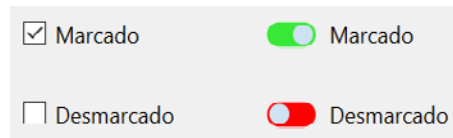


En el caso particular del **framework Qt** la variedad de componentes incluidos es muy amplia, y esta es una de las razones por las que la oferta de controles desarrollados por terceros no es tan abundante como en otras tecnologías.

1.1. Personalizar la apariencia del componente

En algunas ocasiones, la funcionalidad del nuevo componente que necesitamos coincide completamente con la de un control existente, y tan solo un cambio en su apariencia visual puede ser suficiente. Muchas de las tecnologías de interfaz permiten modificar completamente la apariencia visual de un componente, de forma que, aunque su lógica no cambia, el usuario lo percibirá como un control diferente.

Hoy en día es habitual encontrar en las interfaces que utilizamos (sobre todo en las destinadas a dispositivos móviles) un control de tipo interruptor o switch, que permite alternar entre dos estados. La funcionalidad de este control es realmente la misma que la de una **casilla de verificación**, por lo que podríamos cambiar su apariencia para que se comporte visualmente como un interruptor.



1.2. Derivar de un componente existente

En muchas ocasiones, la funcionalidad del componente que queremos crear no coincide exactamente con la de ningún componente existente, pero es muy similar. La solución para esto suele consistir en **crear un componente derivado del que ya hay haciendo las modificaciones pertinentes** (que normalmente son añadir propiedades, señales o ranuras). No siempre necesitaremos derivar directamente de un componente existente, sino que a veces será más conveniente basarlo en alguna clase superior en la cadena de herencia del componente, una que aporte la funcionalidad base que necesitamos para nuestro nuevo control.

IMPORTANTE

Qt dispone de algunas clases abstractas que sirven de base para la creación de algunos componentes. Por ejemplo, de la clase **QAbstractButton** derivan los distintos tipos de botones: **QCheckBox**, **QPushButton**, **QRadioButton** y **QToolButton**. Si nos planteamos diseñar un botón personalizado, podríamos optar por partir directamente de la clase abstracta o de alguna de sus clases derivadas.

1.3. Crear un componente completamente nuevo

Si la funcionalidad de nuestro nuevo componente no está relacionada con la de ningún componente existente, tendremos que crearlo desde cero. En la mayoría de los casos esto implica heredar de una clase base antecesora de todos los componentes (en el caso de Qt, sería la clase **QWidget**).

En el caso de crear un componente totalmente nuevo, deberemos definir su geometría y apariencia visual. En Qt disponemos para ello de la clase **QPainter**, con una gran cantidad de métodos para poder dibujar nuestro control.

1.4. Agregar varios componentes existentes

Es habitual encontrar una última opción para crear nuevos componentes: **agregar varios componentes existentes** (o incluso combinar componentes incluidos en el framework con otros creados por nosotros o por terceros).

Un posible ejemplo de componente agregado sería un cuadro de texto con un indicador del número de caracteres escritos. En este caso deberíamos agregar un cuadro de texto donde el usuario escribiría, y una etiqueta para indicar el número de caracteres.

02 Creación de nuevos componentes por código.



Los componentes son utilizados, entre otras cosas, para ocultar la complejidad del software en partes más manejables; cada uno de ellos la oculta detrás de una interfaz formada por sus propiedades y los métodos a los que se tiene acceso. Pueden introducirse y eliminarse, e incluso ser intercambiados como parte de un todo.

Esto reduce la complejidad del desarrollo software y mejora su mantenimiento, de modo que permite que el mismo código pueda ser reutilizado en diferentes lugares. El resultado es un **bloque de código encapsulado** en una **clase independiente** que pasa a formar parte del banco de piezas disponibles para formar parte de desarrollos más complejos.

2.1. Componentes definidos por el usuario

Para empezar a **desarrollar un componente propio**, lo más fácil es buscar un componente desde el cual partir, de forma que se hereden sus propiedades y métodos y así poder utilizarlos en el componente que queremos crear.

Por ejemplo, si se desea un diálogo personalizado, podemos heredar de `QDialog`; en cambio, si buscamos un componente botón con alguna funcionalidad personalizada, podemos heredar de `QPushButton`. En el caso que nuestro componente no guarde relación estrecha con otros componentes, lo más adecuado será heredar de `QWidget` directamente, que es la clase base desde la que hereda el resto de widgets.

En nuestros componentes podemos utilizar **clases privadas** que nos ayuden a producir un código más organizado y mantenible.

2.2. Ejemplo: componente cronómetro

EJEMPLO

Ahora ilustraremos todo esto con un ejemplo que iremos construyendo paso a paso.

Es posible que en alguna de nuestras aplicaciones podamos necesitar un **componente visual** que sirva para medir el tiempo transcurrido, por ejemplo, el que usamos para la realización de una tarea de clase, el transcurso de una actividad deportiva, etc. Crearemos un componente que posteriormente podemos **agregar a diferentes aplicaciones con interfaz gráfica de usuario**, sin necesidad de volverlo a crear en cada una, sino que reutilizaremos el código generado para su creación.

A. Elección de objeto base

Si pensamos en qué componente utilizar como base, vemos que no hay ninguno al que le podamos agregar funcionalidad de forma rápida y que sirva para nuestro fin. Por lo tanto, partimos de la **clase base QWidget** y utilizamos otros widgets para construir nuestro componente.

B. Clase cronómetro (sin interfaz)

Empezamos definiendo una clase que nos servirá para controlar de forma interna el tiempo transcurrido y el tiempo transcurrido entre pausas, ya que vamos a construir un cronómetro con la posibilidad de pausa. De esta clase obtendremos, cada cierto tiempo, el tiempo transcurrido desde que pusimos en marcha el cronómetro y la diferencia entre el tiempo transcurrido y el pausado en caso de haber utilizado alguna pausa.

La clase utilizada en Qt para controlar el tiempo transcurrido es **QElapsedTimer**. La idea es usar el contador de tiempo transcurrido (`__tiempo_transcurrido`) al poner en marcha el contador. Si mientras esté en marcha queremos pausar la cuenta, usaremos otro contador (`__tiempo_pausado`) para medir el tiempo que el contador ha estado pausado; así mostraremos la diferencia entre ambos para obtener el tiempo correcto del cronómetro.

Para controlar todo esto definiremos su interfaz en la sección siguiente:


```
from PySide6.QtCore import QTime, QElapsedTimer

class Cronometro():
    def __init__(self):
        self.__tiempo_transcurrido = QElapsedTimer()
        self.__tiempo_pausa = QElapsedTimer()
        self.__acumulador = 0

    def reiniciar(self):
        self.__tiempo_transcurrido.restart()
        self.__acumulador = 0

    def getTime(self):
        return QTime(0, 0).addMSecs(self.__tiempo_transcurrido.
elapsed() - self.__acumulador)

    def pausar(self):
        self.__tiempo_pausa.restart()

    def continuar(self):
        self.__acumulador = self.__acumulador + self.__tiempo_pausa.
elapsed()
```

Fíjate en que hemos definido todas las propiedades de la clase anterior como **privadas**, ya que van precedidas de doble guion bajo. De esta forma, desde fuera de la clase no se puede acceder a ellas, solo podemos interactuar con el control del tiempo a través de los métodos reiniciar, pausar y continuar. Con **obtenerTiempo** obtendríamos el tiempo de marcha del cronómetro, independientemente de si ha sido pausado o no.

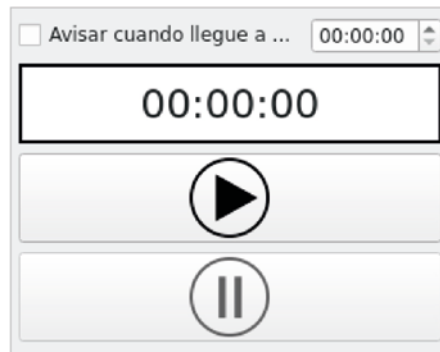
C. Definición de la interfaz del componente

Una vez hemos definido la clase que controla nuestro tiempo, es momento de **definir la interfaz** que usará nuestro componente.

Para mostrar el tiempo transcurrido podemos utilizar una etiqueta (**QLabel**) con el formato hh:mm:ss, donde hh son las horas transcurridas, mm los minutos y ss los segundos. Se podría reescribir el código para que el cronómetro tuviera la posibilidad de contar los milisegundos, pero para el ejemplo que estamos desarrollando lo dejaremos en segundos. Definiremos también un par de botones (**QPushButton**): uno para iniciar/parar y reiniciar la cuenta, y otro para pausar/continuar.

Además, el cronómetro tendrá la posibilidad de avisar, mediante la emisión de una señal definida por código, de que se ha llegado al tiempo marcado en el aviso. Esta señal será emitida cuando el tiempo de aviso y el tiempo del cronómetro coincidan, siempre y cuando el checkbox esté activado.

Para introducir el tiempo, usamos un **QTimeEdit**. La interfaz quedaría como sigue:



Imaginada que el código del componente se encuentra en un archivo llamado `DI_U04_A02_01.py`. En él se importa el archivo de recursos que contiene cada uno de los iconos de play, replay, stop, pause y continue para los botones. Para producir este archivo se ha utilizado la orden siguiente, la cual nos proporciona un archivo de Python a partir del archivo de **recursos.qrc**, escrito en XML:

```
pyside6-rcc DI_U04_A02_02.qrc -o DI_U04_A02_03.py
```

D. Señales definidas por el usuario

Como hemos comentado anteriormente, utilizaremos la emisión de una señal para avisar de que el tiempo se ha cumplido. Para ello, basta con **definir una señal en nuestro componente y emitirla por código** cuando se den las condiciones que deseamos, ya sea un evento del usuario o unas condiciones que se cumplan.

En la definición de la señal, se recibe como parámetro el tipo de información que la acompañará cuando se emita. En el punto en que emitimos la señal daremos valor a esta información definiendo ya su valor. En este caso, junto con la señal vamos a emitir un mensaje indicando que se ha cumplido el tiempo, así que en la definición pasaremos un **str** como argumento.

Esta señal definida mediante código se utilizará igual que cualquier otra señal, es decir, la conectaremos a un slot para ejecutar la funcionalidad deseada cuando se emita.

Mostramos aquí los fragmentos que atañen a la definición y emisión de la señal:

```
from PySide6.QtCore import Signal
...
self.mensaje = Signal(str)
...
self.mensaje.emit("Tiempo límite alcanzado")
```

E. Lógica interna del componente

Ya solo falta **definir la lógica interna del componente**. La idea básica es que definimos un **QTimer**, la **clase Qt** utilizada para implementar temporizadores. Se usa conectando una ranura a su señal timeout, que será emitida a intervalos regulares cada x milisegundos. El número de milisegundos se recibe como argumento en la función start().

Vemos aquí la parte que hace referencia al temporizador en nuestro componente:

```
from PySide6.QtCore import QTimer

...
self.__tiempo = QTimer(self)
self.__tiempo.timeout.connect(self.actualizar_tiempo)
...
self.__tiempo.start(1000)
...
self.__tiempo.stop()
...
```

En nuestro componente definimos un **QTimer privado**, que no podrá ser modificado desde fuera de nuestro componente. Cada segundo actualizará el tiempo mostrado en el display del componente, ya que hemos lanzado el QTimer con el parámetro 1000.

Al actualizar el cronómetro también comprobará si debe emitir la señal de aviso. En el caso que la diferencia entre el tiempo del cronómetro y del aviso sea menor de 200 milisegundos, se lanzará el aviso. Se debe **dejar un pequeño margen** porque pueden producirse retrasos en los procesos que se están ejecutando en el ordenador, y, de no dejar dicho margen, la comparación entre el tiempo actual y el del timeout, puede ser demasiado grande y no lanzar el aviso, sobre todo en ordenadores lentos o con muchos procesos en ejecución.

```
self.__tiempo = QTimer(self)
self.__tiempo.timeout.connect(self.actualizar_tiempo)
@Slot()
def actualizar_tiempo(self):
    crono_actual = self.__cronometro.obtenerTiempo()
    self.etiqueta.setText(
        crono_actual.toString("hh:mm:ss"))
    self.etiqueta.repaint()
# Para que actualice el valor antes de lanzar el aviso
    if self.aviso.isChecked():
        if -200 < self.__tiempo_aviso.msecsTo(crono_actual) < 200:
            self.mensaje.emit("Tiempo límite alcanzado")
```

Por último, necesitamos **representar el estado interno del cronómetro** para implementar las funcionalidades de parar/iniciar y reiniciar, por un lado, y las de pausar/continuar por la otra. Para ello, definimos **cuatro estados**:

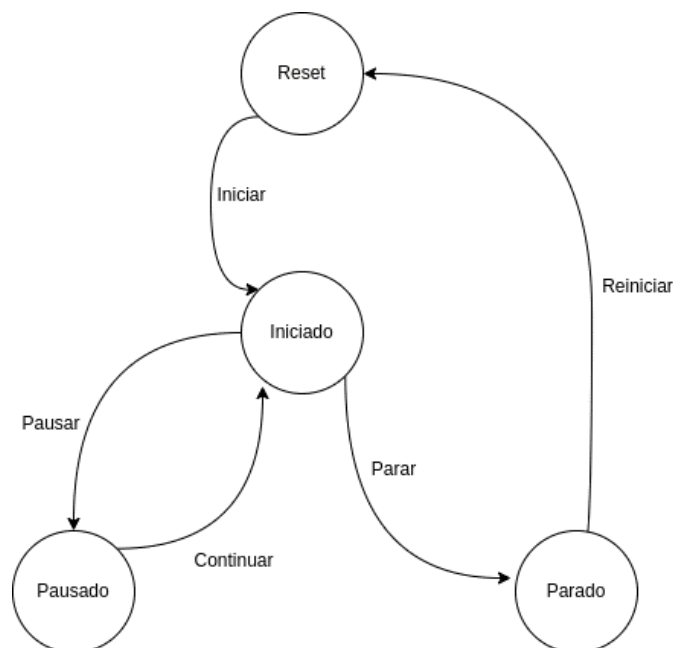
CRONOMETRO_RESET = 0 # Estado inicial

CRONOMETRO_INICIADO = 1

CRONOMETRO_PAUSADO = 2

CRONOMETRO_PARADO = 3

Del estado inicial pasará a iniciado pulsando en iniciar (**play**). Desde este se podrá parar (**stop**) o pausar (**pause**), pasando a los estados parado y pausado, respectivamente. En estado pausado podrá iniciarse de nuevo con continuar (**continue**) y entonces pasará a iniciado. Desde el estado parado, si se pulsa en reiniciar (**restart**), volverá al estado inicial. En el **diagrama de estados** siguiente queda reflejado el anterior comportamiento:



Con esto tenemos completo el código de nuestro componente. **Un componente por sí solo no es útil**, ya que no podemos ejecutarlo, pues solo contiene la definición e implementación de una clase, pero ninguna instanciación de esta. En el apartado siguiente veremos cómo integrarlo en una aplicación.

03 Utilización de los nuevos componentes en aplicaciones

IMPORTANTE

La creación de un nuevo componente añade a nuestro banco de piezas utilizables una posibilidad. Ahora estos nuevos componentes pueden ser utilizados en diferentes aplicaciones, para crear desarrollos más complejos sin tener que reescribir el código. Así podemos reutilizar código escrito por nosotros mismos o por terceros y utilizarlo en nuestras aplicaciones.

3.1. Importación de componentes



VÍDEO LAB

Para consultar el siguiente vídeo sobre Importación de componentes, escanea el código QR o [pulsa aquí](#).

Para usar nuestros componentes, el primer paso siempre será incorporar su código a una aplicación con la instrucción **import** de Python. A partir de ese momento, las **clases** definidas en estos ficheros de código pasarán a estar **disponibles** en nuestra aplicación. Estudiemos un poco el sistema de importación de módulos y paquetes de Python antes de utilizar nuestros componentes.

A. Módulos

DEFINICIÓN

Un **módulo** de Python se define, según el glosario de python.org, como:

«Un objeto que sirve como unidad organizativa del código de Python. Tiene un espacio de nombres que contiene objetos Python arbitrarios. Se cargan mediante el proceso de importación.»

En la práctica, un módulo Python es un archivo .py con código.

Por ejemplo, imaginemos que tenemos un archivo **constantes.py** con el código `PI=3.14159` en su interior. Usaríamos el código siguiente, situado en la misma ruta que el archivo de constantes, para imprimir el valor de la constante por consola:

```
import constantes
print(constantes.PI)
```

B. Paquetes

DEFINICIÓN

Un **paquete** de Python se define, según el glosario de python.org, como:

«Un módulo de Python que puede contener submódulos o, recursivamente, subpaquetes. Técnicamente, un paquete es un módulo de Python con un atributo `__path__`.»

Fíjate en que un paquete no deja de ser un módulo, así que no importa si lo que estamos importando, está estructurado como un módulo o como un paquete. Para un usuario esto es indiferente, ya que **la importación funciona de forma análoga**.

IMPORTANTE

En la práctica, un paquete es una carpeta que contiene archivos de Python y otras carpetas. Para crear un paquete, hay que crear un directorio y, en su interior, un archivo llamado `__init__.py`. Este archivo contiene el código cuando se trata como un módulo. Se puede dejar vacío en el caso de que queramos organizar nuestro código en una estructura de subcarpetas. También puede contener imports implícitos.

Veamos un ejemplo de cómo organizar nuestro código:

```
app.py
idiomas/ ├── africa/
|         ├── __init__.py
|         └── suajili.py
└── europa/
    ├── __init__.py
    ├── español.py
    └── ingles.py
└── __init__.py -> vacío
```

Archivo	Contenido
idioma/__init__.py	(vacío)
idiomas/afrika/__init__.py	(vacío)
idiomas/afrika/suajili.py	saludo = 'Salamu Dunia!'
idiomas/europa/__init__.py	from . import español
idiomas/europa/español.py	saludo = 'Hola mundo!'
idiomas/europa/ingles.py	saludo = 'Hello world!'

Vamos a utilizar este paquete en el archivo app.py. El código quedaría como sigue:

```
from idiomas.afrika import suajili
print(suajili.saludo)
from idiomas import europa
print(europa.español.saludo)
import idiomas.europa.ingles as ingles
print(ingles.saludo)
```

En el primer caso, para utilizar el saludo en suajili debemos hacer el import explícitamente para poder usarlo. En el segundo caso, ya que tenemos el import en idiomas/europa/__init__.py, basta con importar el paquete europa para poder usar el saludo en español. En el último caso, necesitamos importar el módulo ingles.py para poder usar el saludo en este idioma. Si ejecutamos el código, obtenemos el resultado por consola siguiente:

```
Salamu Dunia!
Hola mundo!
Hello world!
```



Esta forma de estructurar y usar los paquetes y módulos puede servir para mantener el código de vuestros componentes organizados de forma lógica y cómoda para utilizarlo en el desarrollo de proyectos.

3.2. Uso de componentes en la aplicación principal

Una vez importado el componente en nuestra aplicación, procedemos a utilizarlo como cualquier otro componente de la librería PySide6. El **primer paso** será **crear las instancias** que necesitemos de este, para, posteriormente, actuar sobre el objeto. Tendremos disponibles todas las propiedades y los métodos públicos que haya heredado de su clase base, más las propiedades y métodos que le hayamos definido en su implementación.

EJEMPLO

A. Ejemplo cronómetro

Continuando con el ejemplo del cronómetro, vamos a incorporar nuestro componente a una aplicación.

Tal y como ya hemos indicado, el primer paso consistirá en importar nuestro componente, que recordamos que se encuentra en nuestros archivos descargables llamados DI_U04_A02_01.py:

```
from DI_U04_A02_01 import CronometroUI
```

Además, necesitamos importar también nuestro archivo de recursos compilado, ya que en el archivo DI_U04_A03_02.qrc hemos definido un recurso conometro.png para cargar nuestro icono de aplicación.

```
import DI_U04_A03_03
```

Con los módulos necesarios incorporados a nuestro código, vamos a utilizar nuestro componente como lo haríamos con cualquier otro componente de la PySide6. En esta ocasión, crearemos una aplicación que esté siempre en el **system tray o bandeja del sistema**. Poco a poco, vamos a ir viendo el código para conseguir este propósito.

Primero creamos la aplicación y la configuramos para que no se cierre al cerrar la última ventana (para que se mantenga en la bandeja del sistema) y le asignamos un icono.

```
app = QApplication([])
app.setQuitOnLastWindowClosed(False)
app.setWindowIcon(QIcon(":/icons/cronometro.png"))
```

Asignamos un icono a la bandeja del sistema y conectamos la **señal activated** para que se muestre/oculte la ventana de ejecución de la aplicación:


```
icon = QIcon(QIcon(":/icons/cronometro.png"))
tray = QSystemTrayIcon()
tray.setIcon(icon)
tray.setVisible(True)
tray.activated.connect(mostrar_ocultar)
```

Seguidamente, instanciamos al **componente cronometroUI** y le asignamos un título. También configuramos que siempre sea visible, para que la aplicación se muestre cuando hagamos clic en su icono en la bandeja del sistema.

```
cronometro.setWindowTitle("Cronómetro PySide6")
cronometro.setWindowFlag(Qt.WindowStaysOnTopHint)
```

Por último, añadimos la posibilidad de terminar la aplicación desde la bandeja del sistema. De otro modo, solo sería posible hacerlo matando el proceso, ya que al cerrar la ventana la aplicación se mantiene en ejecución. Para ello, creamos una acción de salir a la que conectamos el **slot quit** de la aplicación, después añadimos la acción a un menú contextual que aparecerá al hacer clic con el botón secundario del ratón sobre el icono de la aplicación en la bandeja del sistema.

```
accion_salir = QAction("Salir", cronometro)
accion_salir.triggered.connect(app.quit)
menu = QMenu()
menu.addAction(accion_salir)
tray.setContextMenu(menu)
```

B. Uso de la señal del componente

Cuando creamos el componente, definimos una señal que se emite siempre y cuando el tiempo al que llega el cronómetro coincida con el tiempo de aviso y si el checkbox de avisar está marcado. Ahora vamos a usar esa señal.

NOTA

El diseño del componente no define un comportamiento predeterminado al evento de cumplirse el tiempo de aviso, sino que simplemente se emite la señal junto con un mensaje.

Esto nos da total libertad para utilizar esa señal y conectarla al slot que definamos en nuestra aplicación. Nosotros la vamos a utilizar para mostrar un **QMessageBox** con el mensaje que nos envía la señal informando de que el tiempo se ha cumplido; sin embargo, no es obligatorio utilizar este mensaje, podemos usar uno propio si así lo deseamos.

```
...  
  
cronometro.mensaje.connect(mostrar_avisos)  
  
...  
@Slot()  
def mostrar_avisos(mensaje):  
    QMessageBox.information(cronometro, "Cronómetro PySide6", mensaje)
```

3.3. Uso de decoradores en Python

En el anterior apartado, usamos la instrucción **@Slot()** antes de definir el slot. Vamos a ver ahora qué son los **decoradores** y cómo podemos utilizarlos en nuestros componentes para no tener que reescribir el código de las aplicaciones si queremos cambiar la implementación de los componentes.

A. Decoradores

Una función de decorado o un decorador es una función que agrega una nueva funcionalidad a una función que se pasa como argumento. Esto nos permite agregar una nueva funcionalidad a una función existente sin modificar el código de la original ni el código donde esa función se utiliza.

Fíjate en el fragmento de código siguiente:

```
def funcion():  
    print("Funcionalidad original")  
funcion()
```

Si ejecutamos la función, obtendremos por pantalla **"Funcionalidad original"**. ¿Cómo podemos agregar una nueva funcionalidad sin tocar el código de la función original? Aquí es donde entran en juego los decoradores.

Vamos a añadir un decorador a la función original:

```
def decorador(funcion_original):  
    def nueva_funcion():  
        print("Funcionalidad extra")  
        funcion_original()  
    return nueva_funcion  
  
@decorador  
def funcion():  
    print("Funcionalidad original")  
  
funcion()
```

Si ahora ejecutamos de nuevo el código, la salida por consola mostrará ambos mensajes:

```
Funcionalidad extra  
Funcionalidad original
```

IMPORTANTE

La función decoradora recibe como parámetro una función (**funcion_original**) y tiene una función anidada a otra función (**nueva_funcion**). La función original se ejecuta dentro de la nueva para mantener su funcionalidad mientras añade nueva funcionalidad en su llamada. La función decoradora devuelve la nueva función, así que, al ejecutar una función decorada con esta función decoradora, se ejecutará la función original más la función añadida sin cambiar la implementación de la función original.

B. El decorador @SLOT()

Aunque PySide6 permite que cualquier objeto callable de Python, como son las funciones, se use como una **ranura** cuando se conectan señales, a veces es necesario marcar explícitamente un método de Python como una **ranura Qt**. PySide6 proporciona el decorador de funciones Slot() para hacer esto. Además, conectar una señal a un método Python decorado también tiene la ventaja de reducir la cantidad de memoria utilizada y es un poco más rápido, así que es recomendable, aunque no necesario, utilizarlos en las ranuras.

C. El decorador @PROPERTY

Partimos del código siguiente:

```
class Componente:
    def __init__(self, atributo):
        self.atributo = atributo

componente = Componente(10)
print(componente.atributo)
```

Este atributo de instancia es público porque su nombre no tiene un guion bajo para declararlo como protegido, ni un doble guion bajo para declararlo como privado. Dado que el atributo es público, podemos acceder a él directamente desde fuera de la clase para imprimirlo. como es el caso.

Al ejecutar el código, veremos en la consola el texto:

```
10
```

Supongamos que se pide que protejamos este atributo pasándolo a privado y validemos que el valor es un entero positivo antes de asignarlo. ¿Cómo lo hacemos?

Si cambiamos el código y lo pasamos a privado, ya no podremos acceder a él desde fuera de la clase:

```
class Componente:
    def __init__(self, atributo):
        self.__atributo = atributo

componente = Componente(10)
print(componente.__atributo)
```

Al ejecutar el código saldrá un error como el siguiente:

```
AttributeError: 'Componente' object has no attribute '__atributo'.
```

En este punto, lo más probable es que decidas añadir **métodos getters y setters** para leer y asignar el valor del atributo. Pero esto tiene un problema gravísimo.



CLAVES Y CONSEJOS

Si la clase ya está en uso en algún proyecto, el programa fallará en cada línea que lea o asigne el atributo, puesto que anteriormente era público y estamos obligando a los desarrolladores a cambiar su código al cambiar nuestra clase.

Aquí es donde podemos hacer uso del **decorador @property**: cambiaremos nuestra clase sin obligar a cambiar el código de las aplicaciones que hacen uso de ella.

Reescribamos, entonces, nuestra clase con el decorador @property:

```
class Componente:
    def __init__(self, atributo):
        self.__atributo = atributo

    @property
    def atributo(self):
        return self.__atributo

    @atributo.setter
    def atributo(self, nuevo_atributo):
        if nuevo_atributo > 0 and isinstance(nuevo_atributo, int):
            self.__atributo = nuevo_atributo
        else:
            print("Por favor, ingrese un valor entero positivo para el atributo")
```

Ahora no haría falta reescribir el código de la aplicación, ya que, al leer el valor del atributo, se ejecutaría el **método atributo()**, devolviendo el valor del atributo privado. Y al asignar el valor se ejecutaría el **setter atributo(nuevo_valor)**.

Si utilizamos la clase con el código siguiente:

```
componente = Componente(10)
print(componente.atributo)
componente.atributo = -1
print(componente.atributo)
componente.atributo = 20
print(componente.atributo)
```

El resultado sería el siguiente:

```
10
Por favor, ingrese un valor entero positivo para el atributo
10
20
```



CLAVES Y CONSEJOS

Al intentar asignar un valor negativo, el atributo no cambia de valor, sino que simplemente lanza el aviso por consola.

Cuando definimos propiedades de componentes podemos cambiar la implementación interna sin afectar a los programas, por lo que podemos agregar getters y setters que actúan como intermediarios de forma transparente para evitar acceder o modificar los datos directamente.

04 Integración de componentes en herramientas de diseño

En los apartados anteriores hemos visto cómo crear nuevos componentes y cuál es la forma de utilizarlos en nuestras aplicaciones, directamente en el código del programa. Sin embargo, también resulta práctico disponer de esos componentes personalizados cuando diseñamos la interfaz por medio de una herramienta visual de diseño.

En este apartado veremos los dos mecanismos que ofrece Qt Designer para poder diseñar formularios que incluyan componentes personalizados:

- » **Promoción de componentes.**
- » **Creación de complementos (plugins).**

4.1. Promoción de componentes

Es posible que durante el diseño de la interfaz de usuario con Qt Designer no tengamos disponible el componente personalizado que queremos usar. Por ejemplo, puede que el componente no esté finalizado, o que este sea específico para una plataforma y el diseño se esté realizando en otra diferente. En esta situación, podemos utilizar la funcionalidad de **Qt Designer** conocida como **promoción de componentes**.

DEFINICIÓN

La **promoción de componentes** es la que nos permite, como diseñadores, utilizar un componente que sí está disponible en Qt Designer, en representación del componente personalizado que lo no está. De esta forma podremos realizar el diseño de la interfaz, aunque no dispongamos del nuevo control.

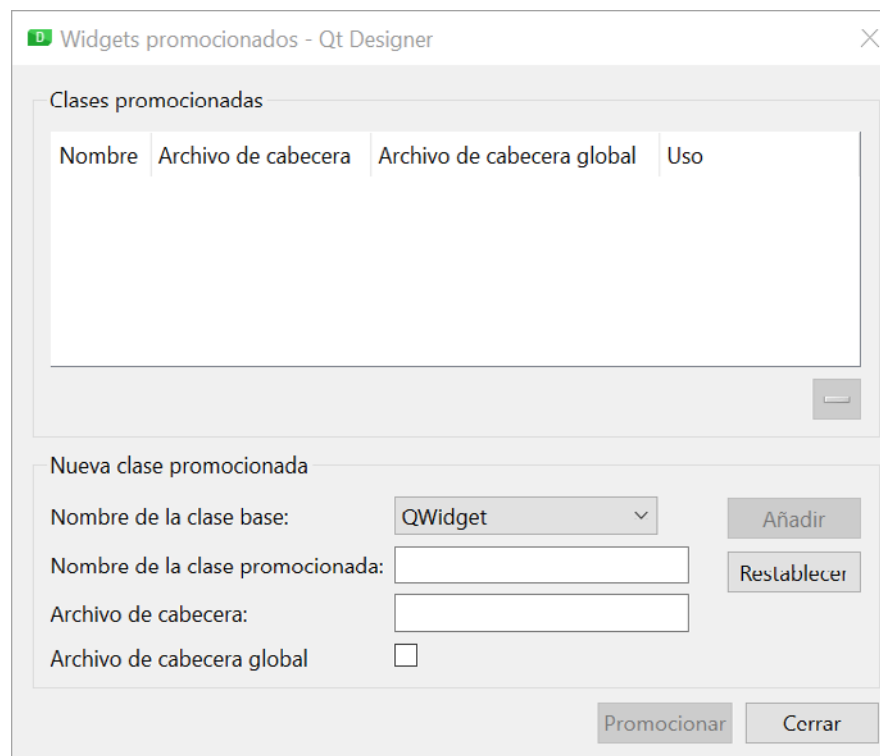
A. Promoción del componente en QT Designer

Lo primero que debemos hacer para utilizar la promoción de componentes es arrastrar el componente que actuará como representante desde la **Caja de widgets de Qt Designer** al formulario. Si el componente personalizado se ha creado derivado de un componente existente, lo ideal será utilizar ese componente base como representante; si no, siempre podremos utilizar como representante un **componente de tipo QWidget** (que encontramos en la sección **Containers** de la Caja de widgets).

IMPORTANTE

Si el componente personalizado que no tenemos disponible es un nuevo tipo de botón que se ha creado derivando de **QPushButton**, el representante será un control de este tipo. Así pues, el comportamiento del componente en el diseñador y en la previsualización será más cercano al que tendrá el nuevo componente cuando se sustituya.

Una vez insertado el componente, deberemos abrir el diálogo **Widgets promocionados** mediante la opción **Promocionar** a del menú contextual del componente.



Widgets promocionados

En este diálogo deberemos completar la información siguiente:

- » **Nombre de la clase promocionada:** indicamos la clase de nuestro componente personalizado.
- » **Archivo de cabecera:** escribimos el nombre del módulo Python que contiene el componente (el nombre del fichero de código sin ninguna extensión).

Una vez completada la información, deberemos pulsar en el botón **Añadir** (que añadirá esta configuración de promoción a Qt Designer, por si queremos reutilizarla) y, después, en el botón **Promocionar** (que aplicará esta configuración de promoción al componente en el que estamos).

Podemos degradar un componente promocionado de nuevo a su tipo inicial con la opción **Degradar** a del menú contextual del componente.

B. Uso de la interfaz diseñada en una aplicación

Cuando el diseño del formulario esté finalizado y se hayan promocionado los componentes necesarios, es momento de **utilizar la interfaz diseñada** en una aplicación. Existen **dos**

mecanismos para hacerlo: convertir el fichero UI a Python con la herramienta **uic**, o cargarlo directamente con la clase **QUiLoader**.

- » Si optamos por la primera opción, la promoción de componentes no introducirá ningún cambio. Cuando la **herramienta uic** genere el código Python asociado al diseño, introducirá la clase promocionada en lugar de la del componente que realmente arrastramos desde la Caja de widgets. Además, añadirá la **orden import** necesaria para cargar el módulo indicado en el diálogo de promoción, por lo que el código generado por **uic** se podrá utilizar directamente desde nuestro programa principal.
- » En el caso de que carguemos el fichero de diseño con la utilidad **QUiLoader** directamente, sí será necesario hacer una modificación en nuestro programa principal. Antes de cargar el fichero UI (con el método **load** de **QUiLoader**) deberemos indicarle al cargador que el formulario tiene un nuevo tipo de componente. Para ello, utilizaremos el método **registerCustomWidget** de **QUiLoader**, pasándole como parámetro la clase asociada al nuevo componente. Esta referencia a la nueva clase nos llevará a tener que añadir la cláusula **import** correspondiente.

4.2. Creación de complementos (plugins)

Cuando tenemos disponible el nuevo componente durante el diseño de la interfaz, Qt Designer ofrece una alternativa a la promoción de componentes, que nos permitirá integrarlo completamente en la herramienta. Para ello, deberemos desarrollar un **complemento (plugin)** asociado a nuestro componente personalizado.

Una vez desarrollado el plugin, nuestro componente aparecerá en la Caja de widgets, como el resto de controles estándar de Qt. El desarrollador podrá arrastrarlo al formulario, de modo que obtenga una representación realista de cómo el componente aparecerá en su aplicación.

A. Actualizar la variable de entorno **PYSIDE_DESIGNER_PLUGINS**

El primer paso para hacer que nuestro componente esté disponible en Qt Designer es **actualizar la variable de entorno **PYSIDE_DESIGNER_PLUGINS****, que contendrá las rutas de todos los plugins que hayamos desarrollado. Si es nuestro primer plugin, antes que nada, deberemos crear la variable de entorno; en el caso de que ya exista, simplemente añadiremos la ruta del nuevo plugin al contenido previo de la variable.

B. Crear el script de registro para el plugin

En el directorio del plugin que hemos incluido en la variable de entorno, además del archivo Python del nuevo componente, será necesario incluir un **script Python** con la información que Qt Designer necesita para registrar el componente.



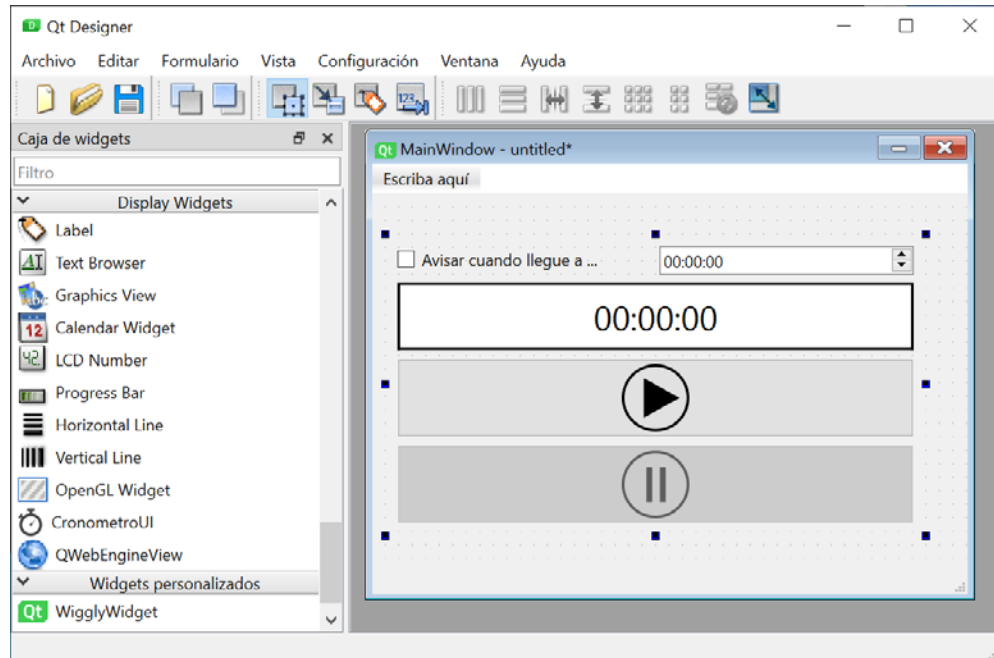
CLAVES Y CONSEJOS

Para que Qt Designer reconozca correctamente el plugin, el nombre del fichero que contiene el script de registro debe estar formado por la palabra «register» seguida del nombre del fichero que contiene el nuevo componente.

En este script haremos uso de la clase **QPyDesignerCustomWidgetCollection**, incluida en el módulo **PySide6.QtDesigner**. Esta clase dispone del método estático **registerCustomWidget** que se encargará de realizar el registro del nuevo componente. Este método recibe los parámetros siguientes:

type	Nombre de la clase de nuestro nuevo componente. Es el único parámetro obligatorio del método, y debe indicarse en primer lugar. Para poder referenciar al nuevo tipo será necesario incluir en el script la cláusula import correspondiente.
xml	Descripción en formato Qt UI de cómo se creará el componente al situarse en un formulario, incluyendo los valores iniciales que deban darse a algunas de sus propiedades.
tool_tip	Texto de ayuda que se mostrará en la Caja de widgets cuando el puntero del ratón se sitúe encima del nuevo componente.
icon	Ruta al icono que se mostrará en la Caja de widgets. Puede ser una ruta local o una referencia a un recurso.
group	Nombre del grupo de controles en la Caja de widgets donde se incluirá el componente. Debe coincidir con el nombre que vemos en Qt Designer.
module	Nombre del módulo Python donde se encuentra el componente (normalmente, el fichero de código sin la extensión). Cuando utilicemos la herramienta uic para generar el código, se insertará en el resultado una cláusula import para este módulo.

En la imagen siguiente podemos ver como el componente se ha integrado en la Caja de widgets de Qt Designer (situado dentro de la sección **Display Widgets**) y la forma en la que se muestra una vez es arrastrado al formulario.



Componente integrado en la Caja de widgets de Qt Designer

C. Uso de la interfaz diseñada en una aplicación

Cuando integramos un nuevo componente en Qt Designer mediante un plugin, el uso del fichero UI resultante en una aplicación no requiere de ninguna acción especial. Se podrá optar por cualquiera de los dos métodos: conversión a código Python con uic o carga directa con **QUILoader**, sin realizar ninguna modificación.

UAX FP